COMP 532
Machine Learning and Bioinspired Optimisation

Lecture 4 – Reinforcement Learning

UNIVERSITY OF LIVERPOOL

Meng Fang
University of Liverpool

Overview

- Problem Solving from Nature
- Reinforcement Leanring
  - Roots of RL
  - Key features of RL
  - Elements of RL
  - Multi-armed bandits
- Deep Learning
- Multi-Agent Learning
- Swam Intelligence

Roots of Reinforcement Learning

Reinforcement Learning draws ideas from multiple disciplines:

- **Psychology** (early 1900s)
  - Trial-and-error learning
  - Behaviorism and reward-driven behavior
- **Control Theory & Operations Research** (1950s)
  - Optimal control
  - Sequential decision making
- **Artificial Intelligence**
  - Planning, search, and learning agents
- **Neuroscience**
  - Dopamine signals as reward prediction errors

Reinforcement Learning in Context

- Reinforcement Learning sits at the intersection of:
  - Artificial Intelligence
  - Psychology
  - Neuroscience
  - Control Theory & Operations Research

- **Note:**
  Deep Neural Networks are *not* a root of RL.
  They are **function approximators** that later enabled **Deep Reinforcement Learning**.

What is Reinforcement Learning?

Reinforcement Learning is:

- An approach to Artificial Intelligence
- Learning from **interaction** with an environment
- **Goal-oriented** learning
- Learning **what to do** (how to act) rather than being told correct actions

Objective:

- Maximize expected cumulative reward:

$$\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]$$

r_0 + 0.1*r_1 +0.1*0.1* r_2+ …

where

r_t is the reward at time t

$\gamma \in [0, 1)$ is the discount factor

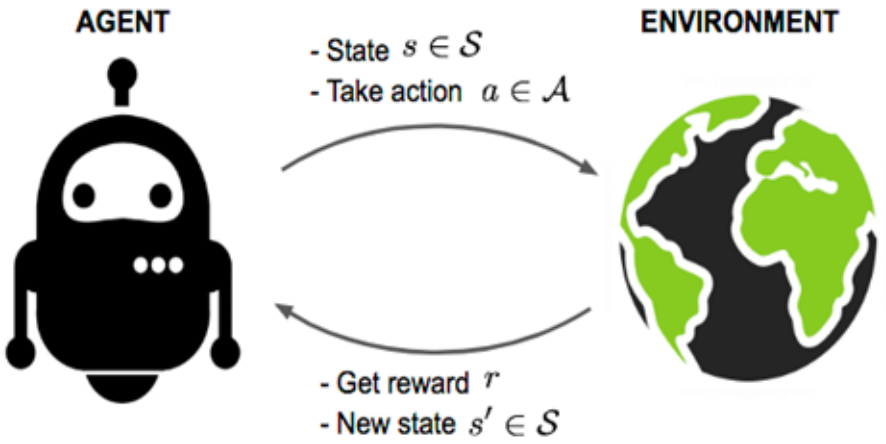
The Standard Agent

A reinforcement learning agent:

- Is **temporally situated**
- Learns and plans continually
- Takes actions to influence the environment
- Operates under **uncertainty and stochasticity**

At each time step:

- Observe state s
- Take action a
- Receive reward r
- Transition to new state s'



Environment Types (1)

Deterministic vs. Stochastic

- **Deterministic**
  - Next state is fully determined by current state and action
- **Stochastic**
  - Next state is drawn from a probability distribution

Examples:

- Crossword puzzle: deterministic
- Taxi driving: stochastic

Environment Types (2)

Fully Observable vs. Partially Observable

- **Fully observable**
  - Agent observes the complete environment state
- **Partially observable**
  - Agent has incomplete or noisy observations

Examples:

- Chess: fully observable
- Taxi driving: partially observable

Environment Types (3)

Episodic vs. Sequential (Continuing)

- **Episodic**
  - Experience divided into episodes
  - Each episode has a terminal state
  - Learning updates do not affect future episodes
- **Sequential (Continuing)**
  - No terminal state
  - Actions have long-term consequences

Examples:

- Maze solving: episodic
- Taxi driving: sequential

Environment Types (4)

Static vs. Dynamic

- **Static**
  - Environment does not change while agent is deliberating
- **Dynamic**
  - Environment may change during decision making

Examples:

- Chess (turn-based): static
- Real-world control: dynamic

Environment Types (5)

Discrete vs. Continuous

- **Discrete**
  - Finite number of states and actions
- **Continuous**
  - States and/or actions are real-valued

Examples:

- Poker: discrete
- Taxi driving: continuous

Environment Types (6)

Single-Agent vs. Multi-Agent

- **Single-agent**
  - Only one decision-making agent
- **Multi-agent**
  - Multiple agents interact, possibly strategically

Examples:

- Crossword puzzle: single-agent
- Taxi driving: multi-agent

Environment types

	Chess	Taxi driving
Fully observable	Yes	No
Deterministic	Strategic	No
Episodic	Yes	No
Static	Yes	No
Discrete	Yes	No
Single agent	No	No

The real world is (of course) partially observable, stochastic, sequential, dynamic, continuous, multi-agent

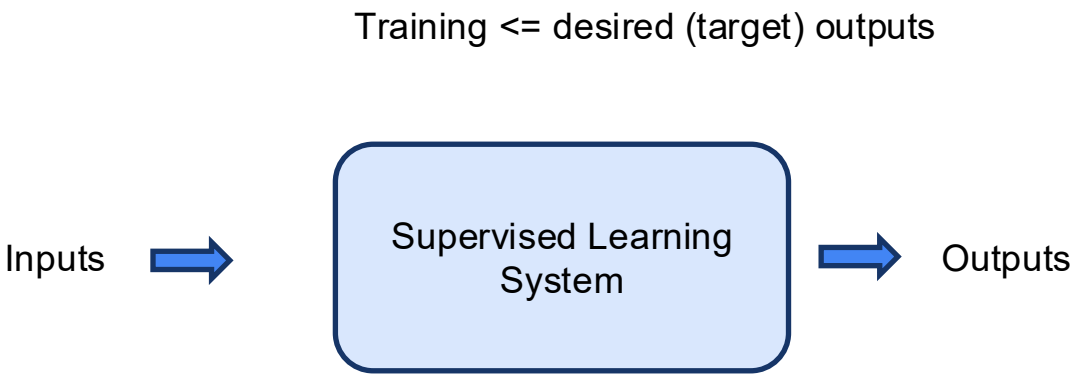
Key Features of Reinforcement Learning

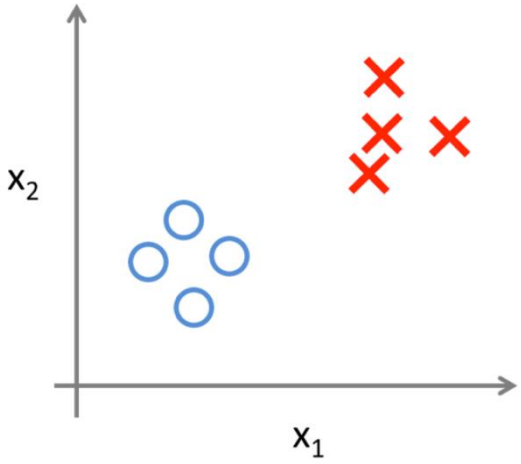
- Agent is **not told which actions to take**
- Learning via **trial and error**
- Rewards may be **delayed**
- Requires balancing:
  - **Exploration** (trying new actions)
  - **Exploitation** (using known good actions)
- Focuses on the **entire agent–environment interaction loop**

Supervised Learning vs. Reinforcement Learning

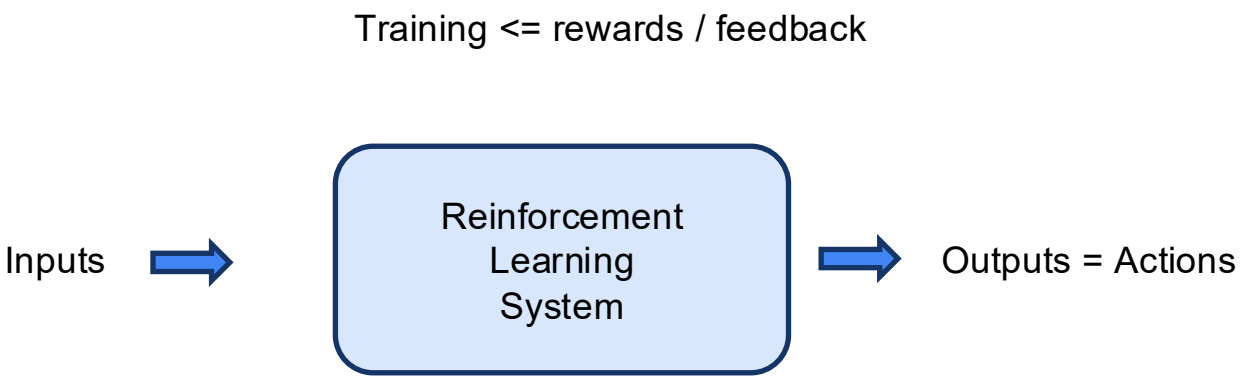
- **Supervised Learning**
  - Correct labels provided
  - Objective defined per data point
  - No delayed consequences
- **Reinforcement Learning**
  - No correct action labels
  - Delayed reward
  - Credit assignment problem
  - Sequential decision making

Supervised Learning





Reinforcement Learning



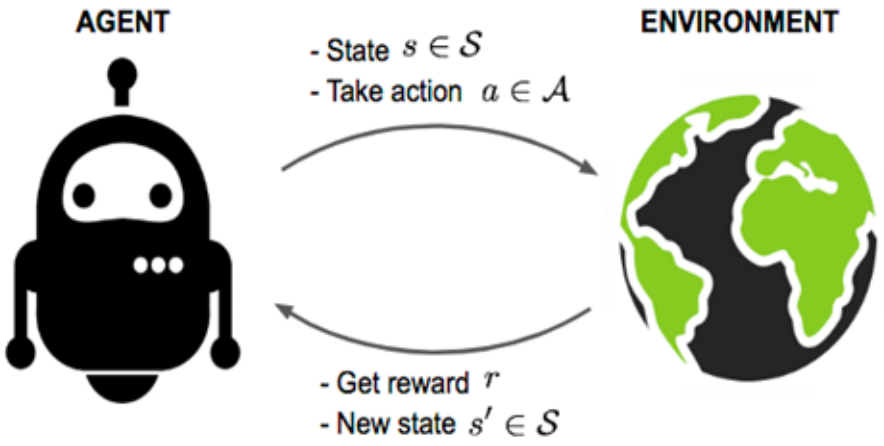
Objective: get as much reward as possible

Elements of Reinforcement Learning

Elements of Reinforcement Learning

action
state

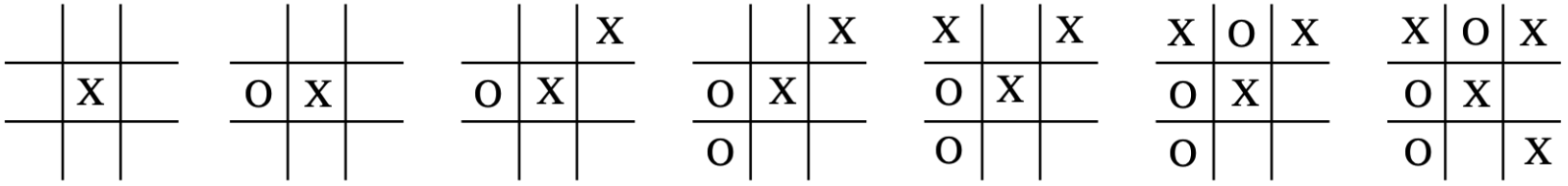
- **Policy** $\pi(a \mid s)$:what to do
  - Mapping from states to actions
- **Reward**: what is immediately good
  - Scalar feedback signal
- **Value function** $V(s)$:what is good in the long run
  - Expected cumulative reward
- **Model**: what follows what
  - Predicts transitions and rewards



An example: Tic-Tac-Toe

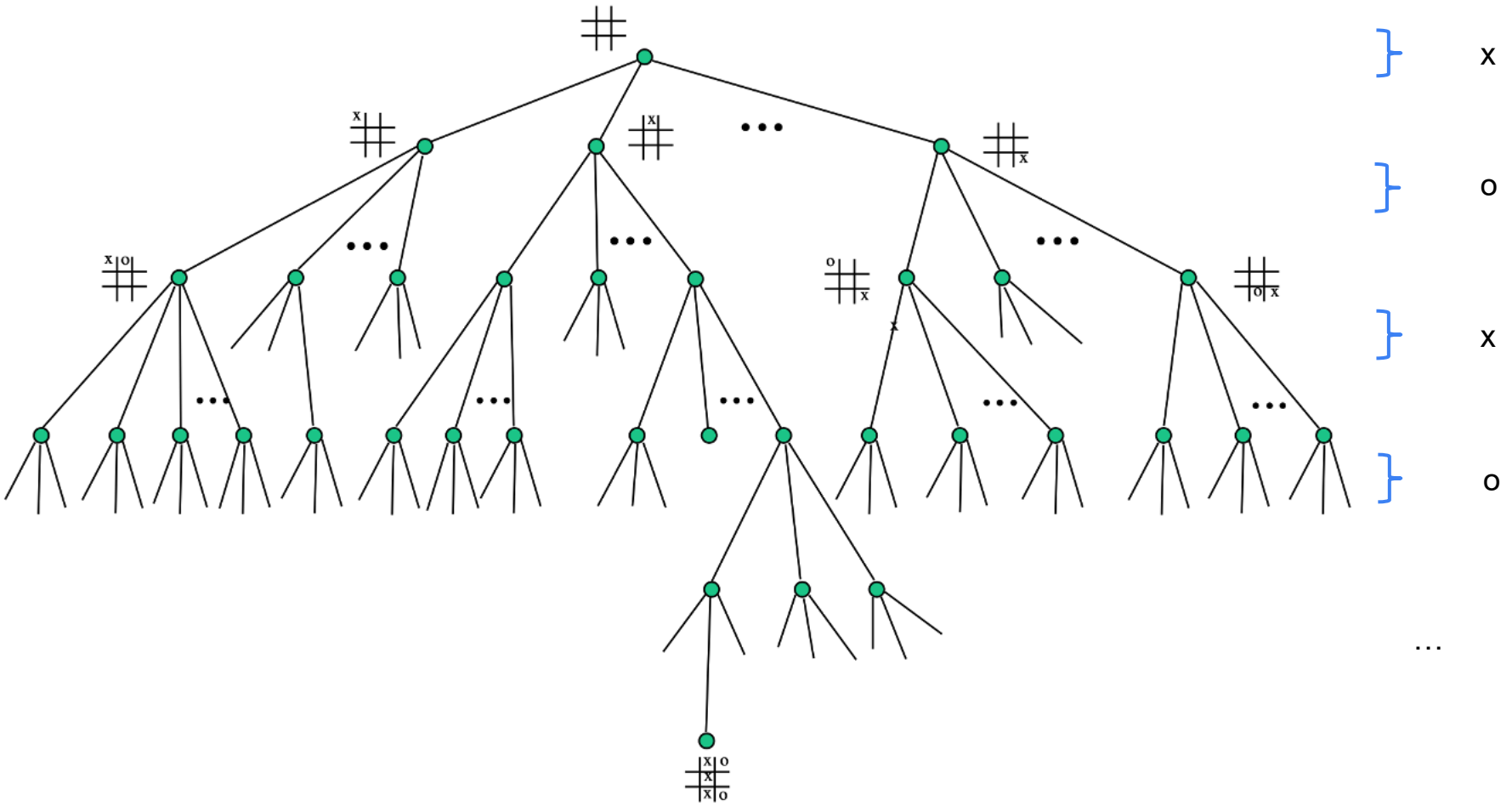
- A simple testbed for reinforcement learning ideas.

- Finite number of states
- Fully observable
- Deterministic
- Episodic
- Two-player, zero-sum game



An example: Tic-Tac-Toe

Tic-Tac-Toe as a State Space
Each board configuration is a state
Legal moves correspond to actions
Terminal states:
Win
Loss
Draw



An RL Approach to Tic-Tac-Toe

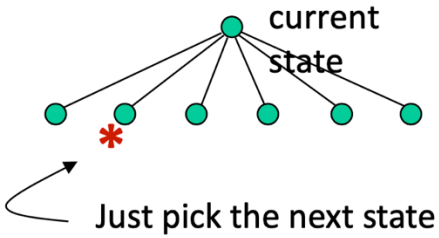
- Maintain a table of state values $V(s)$
- Initialize non-terminal states to 0.5
- Terminal states:
  - Win → 1
  - Loss → 0
  - Draw → 0
- During play:
  - Choose greedy actions most of the time
  - Occasionally explore random moves (ε-greedy)

State	V(s)	
	0.5	?
	0.5	?
…	…	…
	1	win
…	…	…
	0	loss
…	…	…
	0	draw

An RL Approach to Tic-Tac-Toe

State	V(s)	
	0.5	?
	0.5	?
…	…	…
	1	win
…	…	…
	0	loss
…	…	…
	0	draw

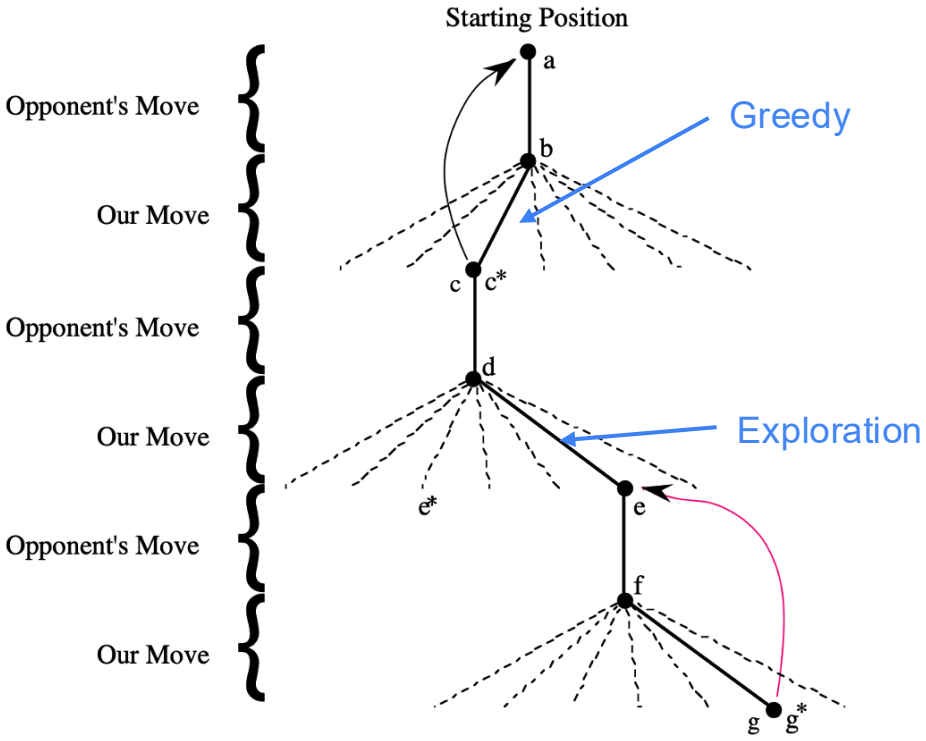
- Now play lots of games
  - To pick our moves, look ahead one step:



Just pick the next state with the highest estimated prob. of winning — the largest $V(s)$; a ***greedy*** move.

But 10% of the time pick a move at random; an ***exploratory move***.

RL method for Tic-Tac-Toe



TD(0) Learning for Tic-Tac-Toe

Let:

- s: state before our move
- s' :state after our move

Update rule (Temporal Difference learning):

- $V(s) \leftarrow V(s) + \alpha[V(s') - V(s)]$

  where:

  $\alpha \in (0, 1]$is the learning rate

A small positive fraction, e.g. $\alpha = 0.1$
the step-size parameter

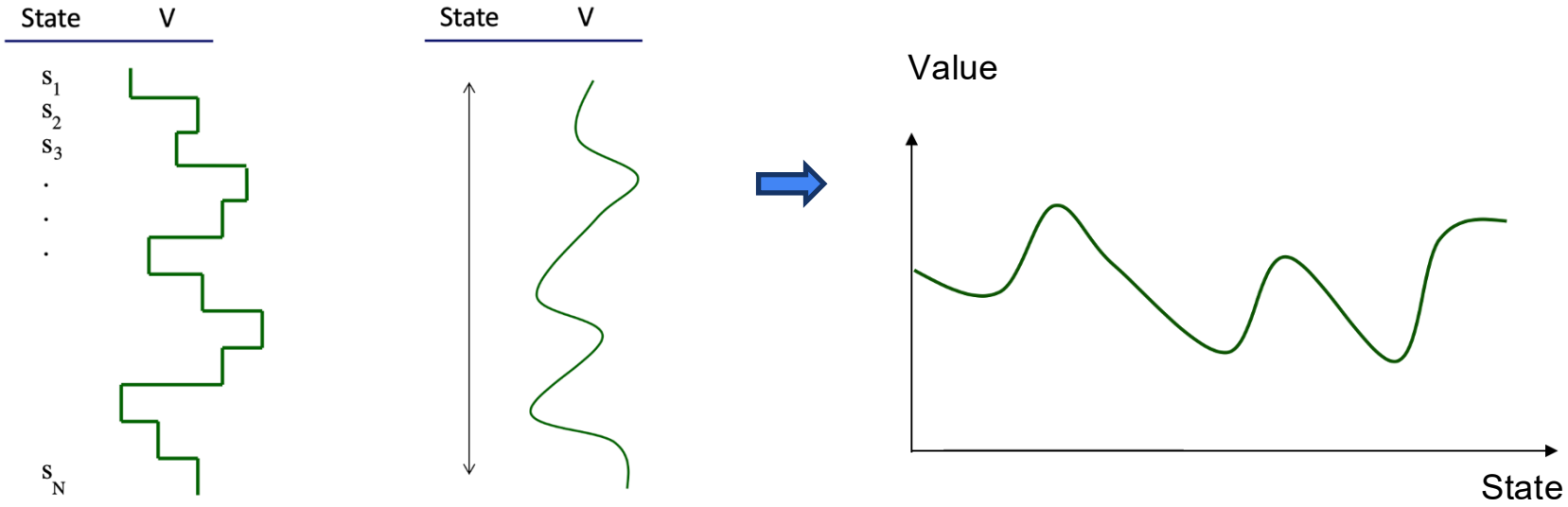
How can we improve the Tic-Tac-Toe player?

Improving the Tic-Tac-Toe Agent

- Exploit board symmetries
- Reduce state space
- Learn from exploratory moves
- Offline learning via self-play
- Opponent modeling

Generalization: Value Function Approximator

- Tabular methods do not scale
- Use function approximation:
  - Linear models
  - Neural networks
- Goal:
- Generalize value estimates across similar states



Why Tic-Tac-Toe Is Too Easy

- Small, finite state space
- Fully observable
- One-step lookahead sufficient
- Real-world problems:
- Large or continuous state spaces
- Partial observability
- Long-term credit assignment

How about other games, like Texas hold 'em?

Home tasks

- Practice to think in a RL way, and identify the four RL elements for:
  - Tic-Tac-Toe
  - Backgammon
  - Poker
  - Chess
  - k-armed bandit
- Read:
  - Sutton & Barto, *Reinforcement Learning*, Chapter 1

Summary

Reinforcement Learning as sequential decision making

- Agent–environment interaction
- Rewards, policies, and value functions
- TD learning through a simple game example
- Motivation for function approximation and deep RL